Homework: Connect Four with Minimax and Alpha-Beta Pruning

1. Objective

Implement a graphical Connect Four game in which a human player competes against an AI agent. Apply the Minimax algorithm and alpha-beta pruning discussed in class, and investigate how search depth affects the AI's playing strength and response time.

2. Game and GUI Requirements

A Java GUI implemented using Swing has been provided. You may extend or customize the provided code in any way you need or develop your own GUI in **Java, Python, or C#**. A graphical user interface is required.

Your implementation must:

- Support a standard board with **6 rows and 7 columns**.
- Support a configurable board with **M rows and N columns**, without requiring changes to the source code.
- Allow a human player to select a column and place a piece.
- Apply gravity: a piece must occupy the lowest available position in the selected column.
- Reject invalid moves, such as placing a piece in a full column.
- Alternate turns between the human player and the AI agent.
- Detect and announce a win when either player connects **four pieces horizontally, vertically, or diagonally**.
- Detect and announce a draw when the board is full and neither player has won.
- Stop accepting moves after the game ends.

For larger boards, the winning condition remains **four connected pieces**. The GUI must accommodate the selected board dimensions.

3. AI Requirements

Implement **Minimax with alpha-beta pruning** to select the AI agent's moves.

Your implementation must:

1. Generate legal moves for the current board.
2. Recognize terminal states: an AI win, a human win, or a draw.
3. Use a heuristic evaluation function to score nonterminal positions when the search reaches its depth limit.

4. Apply alpha-beta pruning to avoid exploring branches that cannot affect the current decision.
5. Allow the search depth to be configured using a variable **d**, through the GUI, a configuration file, or command-line parameters.

For this assignment, **one depth level represents one move by either player**. For example, a search depth of `d = 4` examines up to four individual moves from the current position. A branch must stop earlier if a terminal state is reached.

Evaluate all positions consistently from the AI player's perspective: favorable positions should receive higher scores, and unfavorable positions should receive lower scores. Terminal win and loss scores must take priority over heuristic scores.

The GUI should remain responsive while the AI searches.

4. Experiments and Performance Evaluation

Play against your AI agent and investigate the effect of search depth.

At minimum:

- Test **three different search depths**, such as `d = 2`, `d = 4`, and `d = 6`.
- Play **at least three complete games at each tested depth** on the standard 6 × 7 board.
- Demonstrate that the game also works on at least one larger board, such as 8 × 9.
- Measure the time the AI spends selecting each move. Exclude human thinking time and intentional display delays.
- Record wins, losses, and draws from the AI's perspective.

For a fair comparison, use the same computer and document who moves first. If you vary the starting player, record that information.

Summarize your results using a table such as the following:

Board size	Search depth d	Games played	AI wins	Human wins	Draws	Average AI move time (s)	Maximum AI move time (s)
6 × 7	2						
6 × 7	4						
6 × 7	6						
Larger board	Selected depth						

You may include charts showing search depth versus average response time and search depth versus AI win rate.

5. Short Report

Submit a short report in **PDF format** containing:

1. **Implementation overview:** Explain your game design, Minimax implementation, alpha-beta pruning, and heuristic evaluation function.
2. **Scalability:** Explain how your program supports configurable M × N boards and search depth d.
3. **Experimental setup:** Identify the computer used, tested board sizes and depths, number of games, starting-player arrangement, and timing method.
4. **Results:** Present your measurements and game outcomes in tables and, optionally, charts.
5. **Discussion:** Address the following questions:
    - Can your AI beat a human player? At which tested depths did it win?
    - How long does the AI take to select a move?
    - How does increasing d affect response time and observed playing strength?
    - How does increasing the board size affect performance?
    - What limitations or possible improvements did you identify?

Base your conclusions on your recorded results. A small number of games provides an initial observation of playing strength rather than a reliable estimate of the AI's general winning probability.

6. README Requirements

Include a plain-text file named **README.txt** that explains:

- The programming language, version, and required dependencies.
- How to compile or build the program, if applicable.
- How to launch and play the game.
- How to configure M, N, and d.
- Any command-line parameters, configuration options, or default values.
- At least one complete example of how to run the program.
- Any known issues or limitations.

7. Submission

Submit **one ZIP file** containing:

- All source code, including any modified GUI code.
- Any resources or configuration files needed to run the program.
- The short report in PDF format.
- README.txt.
- The recorded experimental data used in your report.

Your submission must allow the instructor to compile or launch the program by following the README instructions.